

Especificación Técnica Maestra: Sincronización Cretania

Entorno: NeoForge + Velocity Proxy + MySQL | 100% Server-Side

1. Arquitectura y Seguridad

Sistema de persistencia de datos "Server-Side" para inventario, vida, XP y Capabilities (Create, TFC).

Protocolo Seguro

Se utiliza el **Plugin Messaging Channel** (`cretania:sync`) y Velocity Modern Forwarding. No se abren puertos externos para la base de datos (MySQL preferiblemente en 127.0.0.1 o en una red local protegida).

2. Velocity Plugin: Coordinador

```
package com.cretania.velocitysync;

import com.velocitypowered.api.event.Subscribe;
import com.velocitypowered.api.event.connection.PluginMessageEvent;
import com.velocitypowered.api.proxy.messages.MinecraftChannelIdentifier;
import com.velocitypowered.api.proxy.server.RegisteredServer;

public class VelocitySyncPlugin {
    public static final MinecraftChannelIdentifier SYNC_CHANNEL =
        MinecraftChannelIdentifier.from("cretania:sync");

    @Subscribe
    public void onPluginMessage(PluginMessageEvent event) {
        if (!event.getIdentifier().equals(SYNC_CHANNEL)) return;
        if (!(event.getSource() instanceof RegisteredServer)) return; // Prevención de
        exploits

        // TODO: Procesar mensaje de "Guardado Completado" del Servidor A
    }
}
```

3. NeoForge Mod: Guardado y Carga (NBT)

Para evitar bloqueos en el hilo principal (TPS drop), las operaciones hacia MySQL **DEBEN** realizarse de forma asíncrona usando un pool de conexiones como HikariCP.

```
package com.cretania.invsync;

import net.neoforged.neoforge.event.entity.player.PlayerEvent;
import net.neoforged.bus.api.SubscribeEvent;
import net.minecraft.server.level.ServerPlayer;
import net.minecraft.nbt.CompoundTag;

public class InventorySync {

    @SubscribeEvent
    public void onPlayerLogOut(PlayerEvent.PlayerLoggedOutEvent event) {
        if (event.getEntity() instanceof ServerPlayer player) {
            CompoundTag playerData = new CompoundTag();
            player.saveWithoutId(playerData);
            // TODO: Ejecutar asíncronamente (CompletableFuture): Guardar 'playerData'
            en MySQL (LONGTEXT Base64)
            // TODO: Al terminar el guardado -> Enviar mensaje a Velocity vía
            "cretania:sync"
        }
    }

    @SubscribeEvent
    public void onPlayerLogIn(PlayerEvent.PlayerLoggedInEvent event) {
        // TODO: Iniciar estado de bloqueo (SyncStateManager)
        // TODO: Enviar feedback visual Vanilla (Título / Ceguera)
        // TODO: Ejecutar asíncronamente: Consultar MySQL y aplicar NBT cuando esté
        listo
    }
}
```

4. Lógica de Bloqueo y Prevención de Duplicación

Se utiliza un estado asíncrono para congelar al jugador temporalmente mientras MySQL responde. Cancela interacciones para evitar exploits.

```
package com.cretania.invsync.logic;

import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
import net.neoforged.neoforge.event.entity.player.*;
import net.neoforged.neoforge.event.entity.item.ItemTossEvent;
import net.neoforged.bus.api.SubscribeEvent;

public class SyncStateManager {
    public static final ConcurrentHashMap<UUID, Boolean> PENDING_PLAYERS = new
    ConcurrentHashMap<>();

    public static boolean isLocked(UUID uuid) { return
    PENDING_PLAYERS.getDefault(uuid, false); }

    @SubscribeEvent
    public void onPlayerMove(PlayerMoveEvent event) {
        if (isLocked(event.getEntity().getUUID())) event.setCanceled(true);
    }

    @SubscribeEvent
    public void onItemToss(ItemTossEvent event) {
        if (isLocked(event.getPlayer().getUUID())) event.setCanceled(true);
    }

    @SubscribeEvent
    public void onInventoryOpen(PlayerContainerEvent.Open event) {
        if (isLocked(event.getEntity().getUUID())) event.setCanceled(true);
    }
}
```

5. Feedback Visual (100% Server-Side)

```
import net.minecraft.network.protocol.game.ClientboundSetTitleTextPacket;
import net.minecraft.network.chat.Component;
import net.minecraft.world.effect.MobEffectInstance;
import net.minecraft.world.effect.MobEffects;
import net.minecraft.server.level.ServerPlayer;

public class SyncVisuals {
    public static void applyLoadingVisuals(ServerPlayer player) {
        player.connection.send(new
ClientboundSetTitleTextPacket(Component.literal("$6Sincronizando perfil...")));
        player.addEffect(new MobEffectInstance(MobEffects.BLINDNESS, 60, 1, false,
false, false));
    }

    public static void removeLoadingVisuals(ServerPlayer player) {
        player.connection.send(new ClientboundSetTitleTextPacket(Component.empty()));
        player.removeEffect(MobEffects.BLINDNESS);
        player.displayClientMessage(Component.literal("$a;Perfil cargado con éxito!"),
true);
    }
}
```